

MEMORIA TÉCNICA DEL PROYECTO — GREEN CAMPUS MOBILE

Versión del documento: 1.0 **Fecha:** Marzo 2026 **Estado del proyecto:** En desarrollo activo (aplicación no finalizada)

1. DESCRIPCIÓN GENERAL DEL PROYECTO

Green Campus es una aplicación móvil de monitorización y gestión de infraestructuras de edificios universitarios (campus inteligente). Su propósito es centralizar la telemetría de dispositivos IoT distribuidos por el campus, ofrecer dashboards diferenciados por rol de usuario y facilitar la gestión de mantenimiento preventivo.

La aplicación está construida en **Flutter (Dart)** y se comunica con un backend REST mediante HTTPS. La arquitectura sigue un modelo en capas inspirado en los principios de Clean Architecture, con separación estricta entre datos, lógica de negocio y presentación.

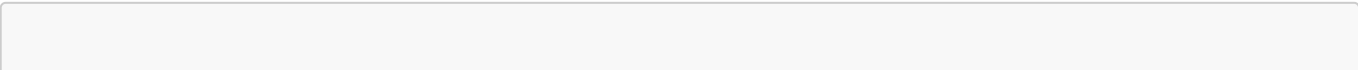
2. STACK TECNOLÓGICO

Componente	Tecnología
Framework UI	Flutter (Material 3)
Lenguaje	Dart
Gestión de estado	ChangeNotifier + ListenableBuilder (nativo)
Almacenamiento seguro	flutter_secure_storage
HTTP Client	package:http (con IOClient custom)
Parseo JSON en hilo	flutter/foundation compute()
Tema visual	Material 3 (ColorScheme.fromSeed)
Fuente tipográfica	JetBrains Mono (para datos técnicos)
Plataforma objetivo	Android (emulador y dispositivo físico)

Nota sobre certificados SSL: Durante el desarrollo se deshabilita la validación de certificados autofirmados para las direcciones **10.0.2.2** (IP estándar del emulador Android hacia localhost) y **localhost**. Esta configuración debe eliminarse en producción.

3. ARQUITECTURA DEL SOFTWARE

La aplicación sigue una arquitectura en 5 capas claramente definidas:



```

lib/
├── core/           → Configuración global (colores, bordes, constantes, config)
├── data/
│   ├── models/    → Entidades del dominio (POJO / DTO)
│   └── services/   → Clientes HTTP (capa de acceso a datos remota)
├── domain/
│   └── validators/ → Lógica de validación pura (sin dependencias externas)
├── logic/          → Controladores de estado (ChangeNotifier) – capa de
aplicación
├── pages/          → Vistas Flutter (UI) – capa de presentación
└── widgets/        → Componentes reutilizables de UI

```

Patrón de gestión de estado

Se utiliza el patrón **ChangeNotifier + ListenableBuilder** nativo de Flutter, sin dependencias de terceros (sin Provider, Riverpod, Bloc). Cada pantalla con estado complejo tiene un controlador dedicado que:

1. Expone estado mediante getters inmutables.
2. Modifica el estado internamente y llama a `notifyListeners()`.
3. La UI escucha con `ListenableBuilder` y se reconstruye solo cuando hay cambios.
4. Libera recursos en `dispose()` (timers, controladores de texto).

4. ESTRUCTURA DETALLADA DE DIRECTORIOS

```

lib/
├── main.dart           → Punto de entrada + sistema de rutas
├── core/
│   ├── app_colors.dart → Paleta de colores centralizada
│   ├── app_borders.dart → Radio de borde global (8px)
│   ├── config/
│   │   └── app_config.dart → URLs base y endpoints de la API
│   └── constants/
│       └── user_roles.dart → Constantes de roles de usuario
├── data/
│   ├── models/
│   │   ├── login_response.dart → DTO de respuesta de login
│   │   ├── technical_data.dart → IotNode + TelemetryData
│   │   └── facility_management_data.dart → HabitabilityStats + ZoneConfort +
PreventiveTask
│   └── services/
│       ├── auth_service.dart → Login, logout, recuperación de
contraseña
│       ├── technical_services.dart → Nodos IoT y telemetría
│       └── facility_management_service.dart → KPIs, zonas, tareas de
mantenimiento
├── profile_service.dart → Actualización de perfil
└── token_storage.dart → Persistencia segura del JWT y datos

```

```

de sesión
|
|— domain/
|   |— validators/
|       |— email_validator.dart           → Validación de formato de email (RFC)
|       |— password_validator.dart       → Reglas de complejidad de contraseña
|
|— logic/
|   |— login/
|       |— login_controller.dart         → Flujo de autenticación
|   |— forgot_password/
|       |— forgot_password_request_controller.dart → Paso 1: solicitud de código
OTP
|   |— new_password_controller.dart       → Paso 2: verificación OTP +
nueva clave
|   |— dashboard/
|       |— technical_dashboard_controller.dart → Estado del dashboard
técnico
|   |— facility_management_controller.dart → Estado del dashboard de
servicios
|   |— profile/
|       |— profile_controller.dart        → Lectura del perfil del usuario
|       |— edit_profile_controller.dart   → Edición de nombre, apellido y
contraseña
|
|— pages/
|   |— splash/
|       |— splash_page.dart              → Pantalla de carga + lógica de
redirección (actualmente comentada en main.dart)
|   |— login/
|       |— login_page.dart               → Formulario de inicio de sesión
|   |— forgot_password/
|       |— request_reset_page.dart       → Paso 1: introducir email
|       |— new_password_page.dart       → Paso 2: OTP + nueva contraseña
|   |— dashboard/
|       |— technical/
|           |— technical_dashboard_page.dart → Dashboard del rol TECNICO
|           |— device_id_search_page.dart  → Listado y búsqueda de
dispositivos
|   |   |— sensors_view/
|       |       |— main_sensor.dart        → Entry point aislado para
preview
|   |   |— view_sensor.dart                → Página de detalle de un
nodo IoT
|   |   |— panel_control_sensor.dart       → Panel de control y salud
del sensor
|   |   |— valor_gauge_sensor.dart        → Widget gauge radial +
configuración
|   |   |— mesual_anual_graficos_sensor.dart → Gráficos
mensuales/anuales del sensor
|   |   |— facility_management/
|       |       |— facility_management_dashboard_page.dart → Dashboard del rol
SERVICIOS_GENERALES
|   |— profile/

```

profile_page.dart	→ Visualización del perfil del usuario
edit_profile_page.dart	→ Edición del perfil
notifications/	
notifications_page.dart	→ Pantalla de notificaciones
(placeholder)	
widgets/	
common_app_header.dart	→ Header/AppBar común con campana y
avatar	
common_app_drawer.dart	→ Drawer lateral adaptativo por rol
primary_button.dart	→ Botón primario estándar con estado de
carga	
email_text_field.dart	→ Campo de email reutilizable
password_text_field.dart	→ Campo de contraseña con visibilidad
toggle	
otp_input_field.dart	→ Entrada OTP de 6 dígitos con
navegación automática	
photo_picker_bottom_sheet.dart	→ Bottom sheet para selección de foto
de perfil	

5. CONFIGURACIÓN GLOBAL (CAPA CORE)

5.1 AppColors (core/app_colors.dart)

Sistema de colores semántico organizado en categorías:

Categoría	Variable	Valor hex
Marca principal	accentGreen	#22C55E
Marca clara	accentGreenLight	#DCFCE7
Fondo de página	bgPage	#FFFFFF
Superficie	surface	#F8F AFC
Texto primario	textPrimary	#1A1A1A
Texto secundario	textSecondary	#64748B
Texto muted	textMuted	#94A3B8
Borde	border	#E2E8F0
Estado ONLINE	statusOnline	#22C55E
Estado WARNING	statusWarning	#F59E0B
Estado OFFLINE	statusOffline	#EF4444

5.2 AppConfig (core/config/app_config.dart)

Centraliza todos los endpoints de la API como constantes estáticas. La URL base (apiUrl) apunta a https://10.0.2.2:3000/v1 (emulador Android). Incluye endpoints parametrizados para operaciones CRUD

sobre nodos IoT mediante el método helper `nodeDetailsEndpoint(String id)`.

Endpoints definidos:

Endpoint	Ruta	Método
Login	<code>/auth/login</code>	POST
Forgot Password	<code>/auth/forgot-password</code>	POST
Reset Password	<code>/auth/reset-password</code>	POST
Logout	<code>/auth/logout</code>	POST
Perfil de usuario	<code>/user/profile</code>	PATCH
Listar nodos IoT	<code>/technical/nodes</code>	GET
Detalle de nodo por ID	<code>/technical/nodes/{id}</code>	GET
Lecturas de un nodo	<code>/technical/nodes/{id}/readings</code>	GET
Promedio mensual de sensor	<code>/technical/nodes/{id}/readings/monthly</code>	GET
Promedio anual de sensor	<code>/technical/nodes/{id}/readings/annual</code>	GET
Tiempo activo mensual por tipo	<code>/technical/nodes/{type}/time-on/monthly</code>	GET
Tiempo activo anual por tipo	<code>/technical/nodes/{type}/time-on/annual</code>	GET
KPIs de habitabilidad	<code>/management/habitability</code>	GET
Estado de zonas	<code>/management/zones</code>	GET
Tareas preventivas	<code>/management/tasks</code>	GET

Nota: Existe un typo en el endpoint `nodeReadingsEndpoint` y `telemetryEndpoint` donde `/tachnical/` debería ser `/technical/`. Esto puede causar errores en tiempo de ejecución.

5.3 UserRoles (`core/constants/user_roles.dart`)

Define tres constantes de string que mapean los roles del sistema:

- **DIRECTIVO** — Acceso al dashboard directivo (en construcción)
- **SERVICIOS_GENERALES** — Acceso al dashboard de gestión de instalaciones
- **TECNICO** — Acceso al dashboard técnico IoT

6. CAPA DE DATOS

6.1 Modelos de Datos

LoginResponse (`data/models/login_response.dart`)

DTO que mapea la respuesta del endpoint `/auth/login`. Campos:

- **jwt** (String): Token de autenticación Bearer

- `email` (String): Correo del usuario
- `role` (String): Rol asignado (ver `UserRoles`)
- `firstName` / `lastName` (String): Nombre y apellido
- `profileImageUrl` (String): URL de la imagen de perfil

IoTNode (`data/models/technical_data.dart`)

Entidad principal del inventario de hardware IoT:

- `id`, `name`, `location`, `type`: Identificación del dispositivo
- `status`: Estado operativo (`ONLINE`, `STANDBY`, `OFFLINE`)
- `battery`: Nivel de batería en porcentaje (0–100)
- `edificio`, `planta`: Ubicación física dentro del campus
- `lastTelemetry` (`TelemetryData?`): Última lectura de sensores (nullable)

TelemetryData (`data/models/technical_data.dart`)

Agregado de lecturas de un nodo en un instante dado:

- `temperature` (`double?`): Temperatura en °C
- `humidity` (`double?`): Humedad relativa en %
- `co2` (`int?`): Concentración de CO2 en ppm
- `energy` (`double?`): Energía consumida en kWh

HabitabilityStats (`data/models/facility_management_data.dart`)

KPIs globales del campus para el dashboard de Servicios Generales:

- `averageTemp`: Temperatura media del campus
- `averageHumidity`: Humedad media
- `averageCo2`: CO2 medio en ppm

ZoneConfort (`data/models/facility_management_data.dart`)

Diagnóstico de confort por zona/aula:

- `id`, `name`: Identificador y nombre de la zona
- `status`: Estado de habitabilidad (`OPTIMO`, etc.)
- `temp`, `humidity`: Condiciones actuales

PreventiveTask (`data/models/facility_management_data.dart`)

Tarea de mantenimiento preventivo:

- `id`, `title`, `location`: Identificación de la tarea
- `daysRemaining`: Días restantes para la ejecución (semáforo visual)
- `priority`: Prioridad (`CRITICAL`, `MEDIUM`, `LOW`)

6.2 Servicios HTTP

Todos los servicios siguen el mismo patrón de construcción:

1. Crean un `HttpClient` con `badCertificateCallback` para desarrollo local.
2. Envuelven el cliente en `IOClient` de `package:http`.
3. Leen el JWT desde `TokenStorage` para autenticar las peticiones.
4. Retornan `http.Response` crudo para que el controlador decida cómo procesarlo.

TokenStorage (`data/services/token_storage.dart`)

Clase estática con doble capa de persistencia:

- **Nivel 1 (caché en memoria):** Variables estáticas `_cachedToken`, `_cachedEmail`, `_cachedFirstName`, `_cachedProfileImage` para lecturas repetitivas sin hit a disco.
- **Nivel 2 (disco cifrado):** `FlutterSecureStorage` con las claves `jwt_token`, `user_role`, `email`, `profile_image_url`, `first_name`, `last_name`.

La operación `saveSession()` escribe simultáneamente en caché y disco. `clearSession()` limpia ambas capas para garantizar un cierre de sesión completo.

7. CAPA DOMAIN — VALIDADORES

EmailValidator

Valida el formato de email mediante expresión regular RFC estándar:

```
^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$
```

Existe código comentado para restringir al dominio `@greencampus.edu` cuando sea necesario activarlo.

PasswordValidator

Valida la complejidad de contraseña con 4 reglas acumulativas:

1. Longitud mínima de 8 caracteres
 2. Al menos una letra mayúscula
 3. Al menos un dígito
 4. Al menos un carácter especial (no alfanumérico)
-

8. CAPA LÓGICA — CONTROLADORES

8.1 LoginController (`logic/login/login_controller.dart`)

Responsabilidades:

- Coordina la llamada a `AuthService.login()`.
- En caso de éxito (HTTP 200), deserializa `LoginResponse` y persiste la sesión en `TokenStorage`.
- Expone `isLoading` y `errorMessage` a la UI.

- Devuelve **bool** al **LoginPage** para que este decida la ruta de navegación según el rol.

Flujo de autenticación:

```
LoginPage → LoginController.login() → AuthService.login() → HTTP POST /auth/login
      ↓ éxito                                     ↓ 200 OK
TokenStorage.saveSession() ← LoginResponse.fromJson(body) ← JSON response
      ↓
Navigator.pushReplacementNamed(role-based route)
```

8.2 ForgotPasswordRequestController

Gestiona el **Paso 1** del flujo de recuperación de contraseña: enviar el email al servidor para recibir el código OTP de 6 dígitos. Si el servidor responde 200, navega al Paso 2 pasando el email como argumento de ruta.

8.3 NewPasswordController

Gestiona el **Paso 2** del flujo de recuperación:

- Recibe el email en el constructor (desde argumentos de ruta).
- Mantiene un timer de cuenta atrás de 60 segundos para el reenvío del código.
- Valida que el código OTP tenga 6 dígitos antes de hacer la petición.
- Llama a **AuthService.resetPassword(email, code, newPassword)**.
- Tras éxito, navega a **/login** eliminando todo el historial de navegación.

8.4 TechnicalDashboardController

Responsabilidades:

- Carga en paralelo datos de perfil (**TokenStorage**) y datos de dashboard (**TechnicalServices**) usando **Future.wait()**.
- Parsea los datos recibidos en un **Isolate** secundario mediante **compute()** para no bloquear el hilo UI.
- Activa un timer de auto-refresco cada **30 segundos** con **_startAutoRefresh()**.
- El refresco silencioso (**showLoading: false**) no dispara el spinner de carga.
- Cancela el timer en **dispose()** para evitar fugas de memoria.

Estado expuesto:

- **isLoading** (bool): Controla el spinner inicial.
- **telemetry** (TelemetryData?): Última lectura agregada de sensores.
- **nodes** (List): Inventario completo de nodos IoT.
- **userName, userEmail, lastName, role, profileImageUrl**: Datos del perfil para el header y drawer.

8.5 FacilityManagementController

Patrón idéntico al **TechnicalDashboardController** pero orientado al rol **SERVICIOS_GENERALES**. Carga tres endpoints en paralelo:

1. **getHabitabilityStats()** → **HabitabilityStats**

2. `getZonesStatus()` → `List<ZoneConfort>`
3. `getPreventiveTasks()` → `List<PreventiveTask>`

8.6 ProfileController

Carga los datos del usuario desde `TokenStorage` (sin llamada HTTP). Expone todos los campos del perfil y un método `refresh()` que delega en `init()` para recargar tras edición.

8.7 EditProfileController

Gestiona el formulario de edición de perfil:

- Pre-carga los `TextEditingController` con los valores actuales de `TokenStorage`.
- Al guardar, llama a `ProfileService.updateProfile()` (PATCH `/user/profile`).
- Si el servidor responde 200, actualiza `TokenStorage` con los nuevos valores de nombre/apellido.
- La contraseña es opcional: solo se envía si el campo no está vacío.
- Señaliza `_success = true` para que la UI vuelva a `ProfilePage` con `result = true`, disparando un `refresh()`.

9. MÓDULO DE AUTENTICACIÓN — FLUJO COMPLETO

9.1 Login

```
SplashPage (inactiva) → LoginPage
├─ Formulario: email + contraseña
├─ Validación local: EmailValidator + PasswordValidator
├─ LoginController.login()
│   └─ HTTP POST /auth/login
│       └─ Éxito → TokenStorage.saveSession() → navegación por rol
│           └─ Error → SnackBar con mensaje del servidor
```

9.2 Recuperación de contraseña (2 pasos)

Paso 1 — RequestResetPage:

```
Usuario introduce email → ForgotPasswordRequestController.requestCode()
├─ HTTP POST /auth/forgot-password
├─ Éxito → Navigator.pushNamed('/new-password', arguments: email)
└─ Error → SnackBar
```

Paso 2 — NewPasswordPage:

```
Usuario introduce código OTP (6 dígitos) + nueva contraseña + confirmación
├─ Validación local: OTP length + PasswordValidator + coincidencia
├─ NewPasswordController.resetPassword(code, newPassword)
│   └─ HTTP POST /auth/reset-password {email, code, newPassword}
```

```
└─ Éxito → Navigator.pushNamedAndRemoveUntil('/login')
└─ Error → SnackBar
└─ Timer de 60s para reenvío del código
```

9.3 Logout

Se implementa en los dashboards y en `SensorDetailPage`. El flujo es:

1. Llamada `AuthService.logout()` con timeout de 4 segundos (fire-and-forget).
2. `TokenStorage.clearSession()` independientemente del resultado del servidor.
3. `Navigator.pushNamedAndRemoveUntil('/login', ...)` para limpiar el historial.

10. MÓDULO TÉCNICO (ROL: TECNICO)

10.1 TechnicalDashboardPage

Dashboard principal con **CustomScrollView + Slivers** para rendimiento óptimo:

Sección 1 — Resumen de Infraestructura: Tres tarjetas estáticas (actualmente con datos hardcodeados):

- Total de nodos: 42
- En línea: 38
- Alertas: 2

Pendiente: Conectar con datos reales calculados desde `_controller.nodes`.

Sección 2 — Sensores en tiempo real: Grid 2x2 con `NeverScrollableScrollPhysics` mostrando los 4 tipos de telemetría:

- Temperatura (°C) — icono naranja
- Humedad (%) — icono azul
- CO2 (ppm) — icono morado
- Energía (kWh) — icono ámbar

La cabecera de sección tiene un chip "LIVE" animado con punto verde pulsante y el intervalo de refresco (30s).

Sección 3 — Inventario de Dispositivos: Lista perezosa (`SliverChildBuilderDelegate`) que solo construye los elementos visibles. Cada ítem muestra:

- Punto de estado semáforo (verde/amarillo/rojo)
- Nombre del nodo + ubicación
- Tipo, estado, nivel de batería
- Toque navega a `/device-details` pasando el objeto `IotNode` como argumento.

Semáforo de estados:

- **ONLINE** → verde (`#22C55E`)
- **STANDBY** → amarillo (`#F59E0B`)
- **OFFLINE** → rojo (`#EF4444`)

10.2 DeviceIdSearchPage

Pantalla de búsqueda de dispositivos con dos modos:

1. **Lista completa:** Carga todos los nodos al entrar con `_loadAllNodes()`. Muestra indicador de carga, estado de error con botón de reintento, o lista vacía según el estado.
2. **Búsqueda por ID:** Campo de texto con validación + botón "Buscar" que llama a `getNodeDetails(id)`. Si existe, abre directamente `SensorDetailPage`.

10.3 SensorDetailPage (view_sensor.dart)

Página de detalle completo de un nodo IoT concreto. Recibe un `IotNode` vía argumentos de ruta. Compone cuatro sub-widgets:

1. **SensorGaugeCard** — gauge radial animado con valor actual
2. **SensorControlPanel** — panel de control y salud del dispositivo
3. **MonthlyActivityCard** — gráfico de actividad de los últimos 30 días
4. **AnnualHistoryCard** — histórico anual del sensor

El tipo de sensor se detecta automáticamente desde `node.type` (string libre de la API):

- Contiene "humedad" → `SensorType.humedad`
- Contiene "calidad" → `SensorType.calidadAire`
- Contiene "matriz" / "termica" → `SensorType.matrizTermica`
- Default → `SensorType.temperatura`

10.4 SensorGaugeCard (valor_gauge_sensor.dart)

Widget altamente parametrizable con gestión propia de ciclo de vida:

Tipos de sensor y configuración (`kSensorConfigs`):

Tipo	Unidad	Max	Rango óptimo	Warning	Danger
Temperatura	°C	60	18–26°C	33°C	48°C
Humedad	%	100	40–60%	70%	90%
Matriz Térmica	°C	150	20–80°C	75°C	112.5°C
Calidad Aire	AQI	500	0–100 AQI	150 AQI	300 AQI

Estados del sensor:

- `low` — Azul (por debajo del umbral mínimo)
- `normal` — Verde (dentro del rango óptimo)
- `warning` — Naranja (por encima del rango óptimo)
- `danger` — Rojo (niveles peligrosos)
- `offline` — Gris (sin conexión)

Mecánica del gauge:

- Arco de 270° dibujado con `CustomPainter` (`_GaugePainter`).

- Zonas de color semitransparentes superpuestas al track de fondo.
- Aguja animada con `AnimationController + Tween + CurvedAnimation(Curves.easeOutCubic)`.
- Auto-polling configurable (default: 30 segundos).
- El callback `fetchValue` inyecta la capa de datos de forma desacoplada.

10.5 SensorControlPanel (panel_control_sensor.dart)

Panel de control y diagnóstico del nodo físico. Incluye:

Controles de acción (actualmente sin lógica):

- Botón "Bajar" — desactivar/reducir
- Botón "Subir" — activar/incrementar
- Botón toggle ON/OFF con animación de color

Tarjetas de salud (con colores semáforo):

- Estado operativo (OK/Error)
- Conectividad (Online/Degraded/Offline)
- Nivel de batería (rojo $\leq 20\%$, amarillo $\leq 50\%$, verde $> 50\%$)
- Señal RSSI en dBm (rojo < -80 , amarillo -80 a -65 , verde ≥ -65)
- Última vez activo (formato relativo: seg/min/h/d)
- Uptime del dispositivo (formato Xh YYm)
- Link Quality en % (rojo $< 50\%$, amarillo $< 75\%$, verde $\geq 75\%$)
- Intervalo de muestreo en segundos

Nota: `SensorHealthData.simulated()` es el origen de datos actual. Pendiente conectar con la API real.

10.6 Gráficos mensuales y anuales (mesual_anual_graficos_sensor.dart)

Módulo con tres clases de widgets visuales para análisis histórico:

1. **DonutSensorGrafico** — Gráfico de donut con indicador de nivel y estadísticas Mín/Prom/Máy.
2. **MonthlyActivityCard** — Gráfico de barras para los últimos 30 días (datos actuales: simulados, pendiente API).
3. **AnnualHistoryCard** — Gráfico histórico anual (datos actuales: simulados, pendiente API).

El sistema de colores de los gráficos tiene 4 niveles: verde/amarillo/naranja/rojo según los rangos del tipo de sensor.

11. MÓDULO DE GESTIÓN DE INSTALACIONES (ROL: SERVICIOS_GENERALES)

FacilityManagementDashboardPage

Dashboard con estructura similar al técnico pero orientado a KPIs de habitabilidad:

Sección 1 — Habitabilidad y Confort: Dos tarjetas de estadística:

- Temperatura media del campus (`HabitabilityStats.averageTemp`)
- CO2 medio en ppm (`HabitabilityStats.averageCo2`)

Pendiente: Falta mostrar la tarjeta de humedad media (`averageHumidity`).

Sección 2 — Plan de Mantenimiento: Lista de `PreventiveTask` con:

- Icono de alerta (rojo si `daysRemaining < 3`, muted si mayor)
- Título de la tarea
- Cuenta atrás en días ("Faltan X días") con color semáforo

12. MÓDULO DE PERFIL

ProfilePage

Vista de solo lectura del perfil. Muestra:

- Avatar circular con imagen de red (o icono placeholder)
- Nombre completo
- Badge del rol con estilo distintivo
- Tarjeta de información personal con filas para: nombre, apellido, email, contraseña (oculta como `.....`), rol

EditProfilePage

Formulario de edición con tres campos:

- **Nombre** (obligatorio)
- **Apellido** (obligatorio)
- **Nueva contraseña** (opcional — vacío = sin cambio)

Al guardar, el controlador llama a `ProfileService.updateProfile()`. Si hay éxito, el `EditProfileController` emite `success = true`, lo que dispara `Navigator.pop(context, true)` y el `ProfilePage` llama a `_controller.refresh()` para actualizar los datos mostrados.

Nota: `PhotoPickerBottomSheet` está implementado como widget reutilizable pero **no está integrado** en `EditProfilePage`. La funcionalidad de cambio de imagen de perfil está pendiente de conectar con `image_picker`.

13. WIDGETS REUTILIZABLES

CommonAppHeader

`SliverToBoxAdapter` que actúa como cabecera de los dashboards. Muestra:

- Botón hamburguesa (abre el Drawer)
- Logo + título "Green Campus"
- Zona derecha: campana de notificaciones (opcional, con badge de conteo) + avatar del usuario (clic navega a `/profile`)

CommonAppDrawer

Drawer lateral adaptativo que cambia su contenido según el rol:

- **DIRECTIVO:** Enlace al dashboard directivo.
- **TECNICO:** `ExpansionTile` con submenús: Vista General, Tipos de Sensor (marcado como "Pronto"), ID de Dispositivo.
- **SERVICIOS_GENERALES:** Enlace al dashboard de servicios.
- **Todos (excepto DIRECTIVO):** Enlace a Notificaciones.
- **Todos:** Header con avatar del usuario (clic → `/profile`), chip del rol, opción de cerrar sesión (roja).

OtpInputField

Campo OTP de 6 dígitos con comportamiento de teclado inteligente:

- Auto-avance al siguiente campo al introducir un dígito.
- Retroceso al campo anterior si el actual está vacío (usando `CallbackShortcuts`).
- Foco automático al primer campo vacío al hacer tap.
- Callback `onCompleted(String code)` al completar los 6 dígitos.

PrimaryButton

Botón de ancho completo (100% del contenedor) con:

- Estado de carga (`isLoading`): Muestra `CircularProgressIndicator` blanco.
- Estado deshabilitado: Color `inputBorder` (`#E2E8F0`).
- Colores: fondo `accentGreen`, texto blanco, sin elevación.

14. SISTEMA DE NAVEGACIÓN

La aplicación usa navegación declarativa con **rutas nombradas** definidas en `main.dart`:

Ruta	Pantalla	Acceso
<code>/</code>	LoginPage	Pública
<code>/login</code>	LoginPage	Pública
<code>/forgot-password</code>	RequestResetPage	Pública
<code>/new-password</code>	NewPasswordPage (args: email)	Pública
<code>/dashboard-directivo</code>	Scaffold placeholder "En construcción"	Rol DIRECTIVO
<code>/dashboard-servicios</code>	FacilityManagementDashboardPage	Rol SERVICIOS_GENERALES
<code>/dashboard-tecnico</code>	TechnicalDashboardPage	Rol TECNICO
<code>/profile</code>	ProfilePage	Autenticado
<code>/edit-profile</code>	EditProfilePage	Autenticado
<code>/notifications</code>	NotificationsPage	TECNICO / SERVICIOS_GENERALES

Ruta	Pantalla	Acceso
/device-id	DeviceIdSearchPage	RoI TECNICO
/device-details	SensorDetailPage (args: lotNode)	RoI TECNICO

Estrategia de limpieza del historial:

- Login exitoso → `pushReplacementNamed` (evita volver atrás al login).
- Logout → `pushNamedAndRemoveUntil('/login', (route) => false)` (elimina todo el historial).
- Nuevas contraseñas exitosas → `pushNamedAndRemoveUntil('/login', ...)`.
- Sub-pantallas (notificaciones, device-id) → `pushNamed` (permite retroceder).

15. SEGURIDAD

Autenticación

- El JWT se almacena en `FlutterSecureStorage` (KeyChain en iOS, EncryptedSharedPreferences en Android).
- Caché en memoria para minimizar lecturas a disco en peticiones frecuentes.
- El token se invalida en el servidor en el logout (HTTP POST `/auth/logout`).
- Si el servidor no responde en el logout, se borra el token local de todas formas.

Validación de datos

- Los formularios usan `GlobalKey<FormState>` con validadores en la capa `domain/validators/`.
- Los campos OTP tienen `LengthLimitingTextInputFormatter(1)` y `FilteringTextInputFormatter.digitsOnly`.

Certificados SSL

- `badCertificateCallback` habilitado solo para `10.0.2.2` y `localhost` en tiempo de desarrollo.
- **Riesgo:** Este callback debe desactivarse o eliminarse en la build de producción.

16. PATRONES Y DECISIONES TÉCNICAS RELEVANTES

Parseo en Isolate secundario

Los controladores de dashboard usan `compute()` para parsear las listas JSON en un hilo secundario, evitando jank en el hilo UI:

```
_nodes = await compute(_parseNodes, results[1].body);
```

Las funciones de parseo (`_parseNodes`, `_parseTelemetry`, etc.) son funciones de nivel superior (no métodos de instancia) para cumplir el requisito de serialización de `compute()`.

Auto-refresco con Timer

Los dashboards se actualizan automáticamente cada 30 segundos mediante `Timer.periodic`. El timer se cancela en `dispose()` para evitar peticiones fantasma tras destruir el widget.

Carga paralela con Future.wait

Las peticiones HTTP independientes se lanzan en paralelo para reducir el tiempo total de carga:

```
final results = await Future.wait([
  _service.getHabitabilityStats(),
  _service.getZonesStatus(),
  _service.getPreventiveTasks(),
]);
```

ThemeData pre-computado

El tema Material 3 se calcula una única vez antes del `runApp()` para evitar recalcular el algoritmo HCT de `ColorScheme.fromSeed` en cada build del widget raíz.

Separación de entry points

El módulo de sensores tiene su propio `main_sensor.dart` que permite ejecutar la vista de detalle de forma aislada (`flutter run -t lib/.../main_sensor.dart`) sin pasar por el proceso de login. Útil para desarrollo y pruebas de UI.

17. ESTADO ACTUAL DEL DESARROLLO Y PENDIENTES

Funcionalidades completadas

- ☒ Sistema completo de autenticación (login, logout, recuperación de contraseña)
- ☒ Persistencia segura de sesión con JWT
- ☒ Sistema de roles con navegación diferenciada
- ☒ Dashboard Técnico con inventario de nodos y telemetría en tiempo real
- ☒ Búsqueda de dispositivos por ID y por lista completa
- ☒ Vista de detalle de sensor con gauge animado y panel de salud
- ☒ Dashboard de Gestión de Instalaciones con KPIs y plan de mantenimiento
- ☒ Módulo de perfil (visualización y edición de nombre/apellido/contraseña)
- ☒ Widgets reutilizables: header, drawer, botones, campos de formulario, OTP
- ☒ Sistema de notificaciones (estructura visual completa)

Funcionalidades en construcción o pendientes

- ☐ **SplashPage:** Implementada pero comentada en `main.dart`. Pendiente activar con lógica de token válido.
- ☐ **Dashboard Directivo:** Pantalla placeholder. Pendiente diseño y endpoints.
- ☐ **Notificaciones:** Solo vista vacía. Pendiente `NotificationsController` + `NotificationsService`.
- ☐ **Gráficos mensuales/anuales:** Widgets implementados con datos simulados. Pendiente integración con endpoints `nodeReadingByMonth` y `nodeReadingByYear`.

- ☐ **Tarjetas de resumen del dashboard técnico:** Los valores "42 nodos", "38 en línea", "2 alertas" son hardcoded. Pendiente calcular desde `_controller.nodes`.
- ☐ **Cambio de foto de perfil:** `PhotoPickerBottomSheet` implementado pero no integrado en `EditProfilePage`. Pendiente añadir `image_picker` y endpoint de subida de imagen.
- ☐ **Tipos de sensor en el Drawer:** La opción "Tipos de Sensor" está marcada como "Pronto" (sin ruta asignada).
- ☐ **Controles del sensor (Subir/Bajar/ON-OFF):** UI implementada sin lógica de negocio ni endpoints.
- ☐ **ZoneConfort en FacilityManagementDashboard:** El modelo y servicio están implementados pero el widget de la lista de zonas no está integrado en la página.
- ☐ **Typo en endpoints:** `tachnical` debe corregirse a `technical` en `telemetryEndpoint` y `nodeReadingsEndpoint`.
- ☐ **Gestión de errores de red:** Actualmente los errores se registran con `debugPrint`. Pendiente sistema centralizado de manejo de errores para la UI.

18. DEPENDENCIAS EXTERNAS IDENTIFICADAS

Paquete	Uso
<code>flutter/material.dart</code>	Framework UI principal
<code>package:http</code>	Cliente HTTP
<code>package:http/io_client</code>	Wrapper para configurar HttpClient con SSL
<code>flutter_secure_storage</code>	Almacenamiento cifrado del JWT
<code>flutter/foundation</code>	<code>compute()</code> para parseo en Isolate
<code>dart:async</code>	Timer.periodic para auto-refresco
<code>dart:convert</code>	<code>jsonDecode</code> / <code>jsonEncode</code>
<code>dart:io</code>	HttpClient y X509Certificate
<code>dart:math</code>	Cálculos trigonométricos del gauge radial

Dependencias pendientes de integrar:

- `image_picker` — Para selección de foto de perfil (widget ya preparado)
-

19. CASOS DE USO IMPLEMENTADOS

Los casos de uso se organizan por actor. Se consideran implementados aquellos que tienen flujo completo desde la UI hasta la llamada HTTP o la lectura de datos locales, aunque el backend pueda estar aún en desarrollo.

Actor: USUARIO NO AUTENTICADO (cualquier rol antes del login)

CU-01 — Iniciar sesión

Descripción: El usuario introduce sus credenciales para acceder a la aplicación y ser redirigido al dashboard correspondiente a su rol.

Precondición: El usuario no tiene sesión activa. La aplicación muestra `LoginPage`.

Flujo principal:

1. El usuario introduce su correo electrónico y contraseña.
2. El sistema valida el formato del email (regex RFC estándar) y la complejidad de la contraseña (mínimo 8 caracteres, mayúscula, número y carácter especial).
3. Si la validación local es correcta, `LoginController` realiza POST a `/auth/login` con las credenciales en JSON.
4. El servidor responde con HTTP 200 y un cuerpo JSON que incluye `jwt`, `role`, `email`, `firstName`, `lastName` y `profileImageUrl`.
5. El sistema persiste todos estos campos en `FlutterSecureStorage` mediante `TokenStorage.saveSession()`.
6. El sistema lee el rol almacenado y redirige con `pushReplacementNamed` a la ruta correspondiente: `/dashboard-directivo`, `/dashboard-servicios` o `/dashboard-tecnico`.

Flujo alternativo — credenciales incorrectas:

- El servidor responde con un código distinto de 200. El sistema muestra un `SnackBar` rojo con el mensaje `body['message']` del servidor, o el texto genérico "Error de acceso" si no viene mensaje.

Flujo alternativo — sin conexión:

- Se lanza una excepción (timeout o error de red). El sistema muestra el `SnackBar` con "No se pudo conectar con el servidor".

Postcondición: La sesión queda persistida localmente. El usuario ve su dashboard.

CU-02 — Solicitar código de recuperación de contraseña

Descripción: El usuario que ha olvidado su contraseña introduce su email para recibir un código OTP de 6 dígitos.

Precondición: El usuario está en `LoginPage` y pulsa "¿Olvidaste tu contraseña?".

Flujo principal:

1. El sistema navega a `RequestResetPage`.
2. El usuario introduce su email.
3. El sistema valida el formato del email localmente.
4. `ForgotPasswordRequestController` realiza POST a `/auth/forgot-password` con el email.
5. El servidor responde HTTP 200 (indica que el email fue enviado).
6. El sistema navega a `NewPasswordPage` pasando el email como argumento de ruta.

Flujo alternativo — email no registrado u otro error del servidor:

- El servidor responde con un código distinto de 200. Se muestra **SnackBar** con el mensaje de error.

Postcondición: El usuario llega al paso 2 con su email disponible para el siguiente CU.

CU-03 — Restablecer contraseña con código OTP

Descripción: El usuario introduce el código OTP recibido por email y establece una nueva contraseña.

Precondición: El usuario ha completado CU-02 y se encuentra en **NewPasswordPage** con su email disponible.

Flujo principal:

1. El usuario introduce el código OTP de 6 dígitos en el campo **OtpInputField**.
2. El usuario introduce la nueva contraseña y la confirmación.
3. El sistema valida localmente: longitud del código (= 6), complejidad de la nueva contraseña y coincidencia entre ambas contraseñas.
4. **NewPasswordController** realiza POST a **/auth/reset-password** con **{email, code, newPassword}**.
5. El servidor responde HTTP 200.
6. El sistema muestra un **SnackBar** verde con "¡Contraseña actualizada!" y navega a **/login** eliminando todo el historial de navegación.

Flujo alternativo — código incorrecto o expirado:

- El servidor responde con un código distinto de 200. Se muestra **SnackBar** rojo con el mensaje del servidor.

Flujo alternativo — reenvío del código:

- El usuario espera a que el timer de 60 segundos llegue a cero. El texto cambia a "Ya puedes reenviar el código". El reenvío efectivo (llamada HTTP) está pendiente de implementar.

Postcondición: La contraseña queda actualizada en el servidor. El usuario puede iniciar sesión con la nueva clave.

Actor: TÉCNICO (rol **TECNICO**)

CU-04 — Consultar resumen de infraestructura IoT

Descripción: El técnico accede a su dashboard y visualiza un resumen del estado global de la red de dispositivos IoT.

Precondición: El técnico ha iniciado sesión. Se encuentra en **TechnicalDashboardPage**.

Flujo principal:

1. Al cargar la página, **TechnicalDashboardController.init()** lanza en paralelo la carga del perfil de usuario y la petición de datos al backend.
2. El sistema realiza en paralelo GET a **/technical/nodes** (inventario) y GET al endpoint de telemetría.

3. Los datos JSON se parsean en un Isolate secundario mediante `compute()`.
4. El sistema muestra tres tarjetas de resumen: total de nodos, nodos en línea y alertas activas.
5. El auto-refresco se activa cada 30 segundos de forma silenciosa (sin spinner).

Postcondición: El técnico dispone de una visión global actualizada del estado de la infraestructura.

CU-05 — Monitorizar sensores en tiempo real

Descripción: El técnico visualiza los valores actuales de los cuatro tipos de métricas ambientales del campus.

Precondición: El dashboard técnico ha cargado los datos de telemetría.

Flujo principal:

1. El sistema renderiza un grid 2x2 con las tarjetas de: Temperatura (°C), Humedad (%), CO2 (ppm) y Energía (kWh).
2. Cada tarjeta muestra el valor actual extraído del objeto `TelemetryData` más reciente.
3. La cabecera del grid muestra un chip "LIVE" con un punto verde pulsante y el intervalo de refresco ("30s").
4. Los datos se actualizan automáticamente cada 30 segundos sin intervención del usuario.
5. El usuario puede forzar una actualización arrastrando hacia abajo (pull-to-refresh mediante `RefreshIndicator`).

Postcondición: El técnico dispone de los valores más recientes de las métricas ambientales.

CU-06 — Consultar inventario de dispositivos IoT

Descripción: El técnico visualiza la lista completa de nodos IoT registrados en el sistema con su estado operativo.

Precondición: El dashboard técnico ha cargado la lista de nodos.

Flujo principal:

1. El sistema renderiza una lista perezosa (lazy) de los nodos IoT recibidos del servidor.
2. Cada ítem de la lista muestra: punto de estado semáforo (verde/amarillo/rojo según `ONLINE`/`STANDBY`/`OFFLINE`), nombre del nodo, ubicación, tipo, estado textual y nivel de batería.
3. La cabecera de la sección indica el número total de nodos ("X nodos").
4. Al pulsar cualquier nodo, el sistema navega a la vista de detalle de ese dispositivo (CU-07).

Postcondición: El técnico tiene visibilidad del estado operativo de cada dispositivo en la red.

CU-07 — Consultar detalle de un nodo IoT específico

Descripción: El técnico accede a la vista completa de un nodo concreto, incluyendo su valor actual de sensor, estado de salud y datos históricos.

Precondición: El técnico ha pulsado un nodo en el inventario (CU-06) o ha buscado uno por ID (CU-08). El objeto `IotNode` se pasa como argumento de ruta.

Flujo principal:

1. `SensorDetailPage` recibe el `IotNode` y determina el tipo de sensor a partir del campo `type` de la API.
2. El sistema muestra el **gauge radial animado** (`SensorGaugeCard`) con el valor actual del sensor, su unidad, el estado semáforo y el rango óptimo.
3. El gauge se refresca automáticamente cada 30 segundos mediante polling interno.
4. El sistema muestra el **Panel de Control** (`SensorControlPanel`) con: estado operativo, conectividad, nivel de batería, señal RSSI, calidad de enlace, último contacto, uptime e intervalo de muestreo.
5. El sistema muestra las tarjetas de **actividad mensual** y **histórico anual** del sensor.
6. El técnico puede interactuar con los botones "Subir", "Bajar" y "ON/OFF" del panel de control.

Flujo alternativo — error al obtener el valor del sensor:

- El gauge muestra el estado `offline` con icono de nube desconectada y botón "Reintentar".

Nota: Los controles ON/OFF/Subir/Bajar tienen la UI implementada pero sin lógica de negocio ni endpoints asociados. Los gráficos históricos usan datos simulados.

Postcondición: El técnico tiene información completa del estado actual e histórico del dispositivo seleccionado.

CU-08 — Buscar un dispositivo IoT por ID

Descripción: El técnico localiza un nodo específico introduciendo directamente su identificador único.

Precondición: El técnico accede a `DeviceIdSearchPage` desde el drawer (menú "ID de Dispositivo").

Flujo principal:

1. Al entrar, el sistema carga automáticamente la lista completa de todos los nodos disponibles.
2. El técnico introduce un ID en el campo de búsqueda (ej. `SEN-042`) y pulsa "Buscar" o la tecla de búsqueda del teclado.
3. El sistema valida que el campo no esté vacío.
4. `TechnicalServices.getNodeDetails(id)` realiza GET a `/technical/nodes/{id}`.
5. Si el servidor responde HTTP 200, el sistema construye el objeto `IotNode` y navega a `/device-details`.
6. Si el servidor responde un código distinto de 200, se muestra un `SnackBar` rojo con "Dispositivo no encontrado".

Flujo alternativo — navegar desde la lista:

- El técnico puede pulsar directamente cualquier nodo de la lista completa cargada al inicio, sin necesidad de introducir el ID manualmente.

Flujo alternativo — error de red:

- La lista muestra un estado de error con icono `wifi_off` y un botón "Reintentar" que re-ejecuta la carga.

Postcondición: El técnico accede a la vista de detalle del dispositivo buscado (CU-07).

Actor: SERVICIOS GENERALES (rol `SERVICIOS_GENERALES`)

CU-09 — Consultar KPIs de habitabilidad del campus

Descripción: El responsable de servicios generales visualiza los indicadores globales de confort ambiental del campus en tiempo real.

Precondición: El usuario ha iniciado sesión con el rol `SERVICIOS_GENERALES`. Se encuentra en `FacilityManagementDashboardPage`.

Flujo principal:

1. Al cargar la página, `FacilityManagementController.init()` lanza en paralelo tres peticiones HTTP.
2. El sistema realiza GET a `/management/habitability` y recibe `HabitabilityStats`.
3. El sistema muestra dos tarjetas de KPI: **Temperatura media** del campus (°C) y **CO2 medio** (ppm).
4. El auto-refresco se activa cada 30 segundos de forma silenciosa.
5. El usuario puede forzar la actualización con pull-to-refresh.

Postcondición: El responsable dispone de los KPIs ambientales más recientes del campus.

CU-10 — Consultar el plan de mantenimiento preventivo

Descripción: El responsable de servicios generales revisa las tareas de mantenimiento programadas y el tiempo restante para cada una.

Precondición: El usuario se encuentra en `FacilityManagementDashboardPage` con los datos ya cargados.

Flujo principal:

1. El sistema realiza GET a `/management/tasks` y recibe una lista de `PreventiveTask`.
2. El sistema muestra la lista de tareas con: título, ubicación y cuenta atrás en días ("Faltan X días").
3. Las tareas con menos de 3 días restantes se marcan con el icono y texto en rojo (`statusOffline`). El resto en color muted.
4. La cabecera de la sección indica el número total de tareas ("X tareas").

Postcondición: El responsable conoce qué tareas requieren atención inmediata y su prioridad temporal.

Actor: CUALQUIER USUARIO AUTENTICADO

CU-11 — Cerrar sesión

Descripción: El usuario autenticado cierra su sesión de forma segura, invalidando el token en el servidor y eliminando los datos locales.

Precondición: El usuario está en cualquier pantalla que incluya el `CommonAppDrawer` (dashboards, vista de detalle de sensor).

Flujo principal:

1. El usuario abre el drawer lateral y pulsa "Cerrar sesión".
2. El sistema realiza POST a `/auth/logout` con el JWT en la cabecera `Authorization: Bearer {token}`, con un timeout de 4 segundos.
3. Independientemente del resultado de la petición, el sistema ejecuta `TokenStorage.clearSession()`, que borra el token y todos los datos de sesión tanto de la caché en memoria como del almacenamiento cifrado.
4. El sistema navega a `/login` eliminando completamente el historial de navegación con `pushNamedAndRemoveUntil`.

Flujo alternativo — servidor no responde:

- La petición de logout lanza una excepción (timeout o error de red). El sistema ignora el error y continúa con la limpieza local (paso 3).

Postcondición: El JWT queda invalidado en el servidor. No hay datos de sesión en el dispositivo. El historial de navegación está limpio: el botón "atrás" no puede devolver al usuario a ninguna pantalla autenticada.

CU-12 — Visualizar el perfil de usuario

Descripción: El usuario consulta la información personal asociada a su cuenta.

Precondición: El usuario está autenticado. Accede a `/profile` desde el avatar del header o desde el drawer.

Flujo principal:

1. `ProfileController.init()` lee en paralelo desde `TokenStorage`: nombre, apellido, email, rol e imagen de perfil.
2. El sistema muestra el avatar circular (imagen de red si existe, o icono placeholder).
3. El sistema muestra el nombre completo, un badge con el rol del usuario y una tarjeta con los campos: nombre, apellido, email, contraseña (oculta como `••••••••`) y rol.
4. El usuario puede pulsar "Editar Perfil" o el icono de edición del header para acceder a CU-13.

Postcondición: El usuario dispone de una vista completa y actualizada de sus datos de perfil.

CU-13 — Editar datos del perfil

Descripción: El usuario modifica su nombre, apellido y/o contraseña.

Precondición: El usuario ha accedido a `EditProfilePage` desde la pantalla de perfil (CU-12).

Flujo principal:

1. El sistema pre-carga los campos "Nombre" y "Apellido" con los valores actuales leídos de `TokenStorage`.
2. El usuario modifica los campos deseados. El campo de contraseña se deja vacío si no se desea cambiar.
3. El usuario pulsa "Guardar Cambios" o el icono de confirmación del header.
4. El sistema valida que nombre y apellido no estén vacíos.
5. `EditProfileController.saveChanges()` realiza PATCH a `/user/profile` con `{firstName, lastName}` y opcionalmente `{password}` si el campo no está vacío.
6. El servidor responde HTTP 200.
7. El sistema actualiza `TokenStorage` con el nuevo nombre y apellido.
8. El sistema navega de vuelta a `ProfilePage` con `result = true`, lo que dispara un `refresh()` para mostrar los datos actualizados.

Flujo alternativo — error del servidor:

- El servidor responde con un código distinto de 200. Se muestra `SnackBar` rojo con "No se pudieron guardar los cambios. Inténtalo de nuevo."

Flujo alternativo — error de red:

- Se lanza una excepción. Se muestra `SnackBar` rojo con "Error de conexión. Comprueba tu red."

Postcondición: Los nuevos datos de nombre/apellido y/o contraseña quedan actualizados en el servidor y en el almacenamiento local del dispositivo.

CU-14 — Consultar notificaciones del sistema

Descripción: El usuario accede a la pantalla de notificaciones del sistema para ver alertas generadas.

Precondición: El usuario tiene rol `TECNICO` o `SERVICIOS_GENERALES`. Accede a `/notifications` desde la campana del header o desde el drawer.

Flujo principal:

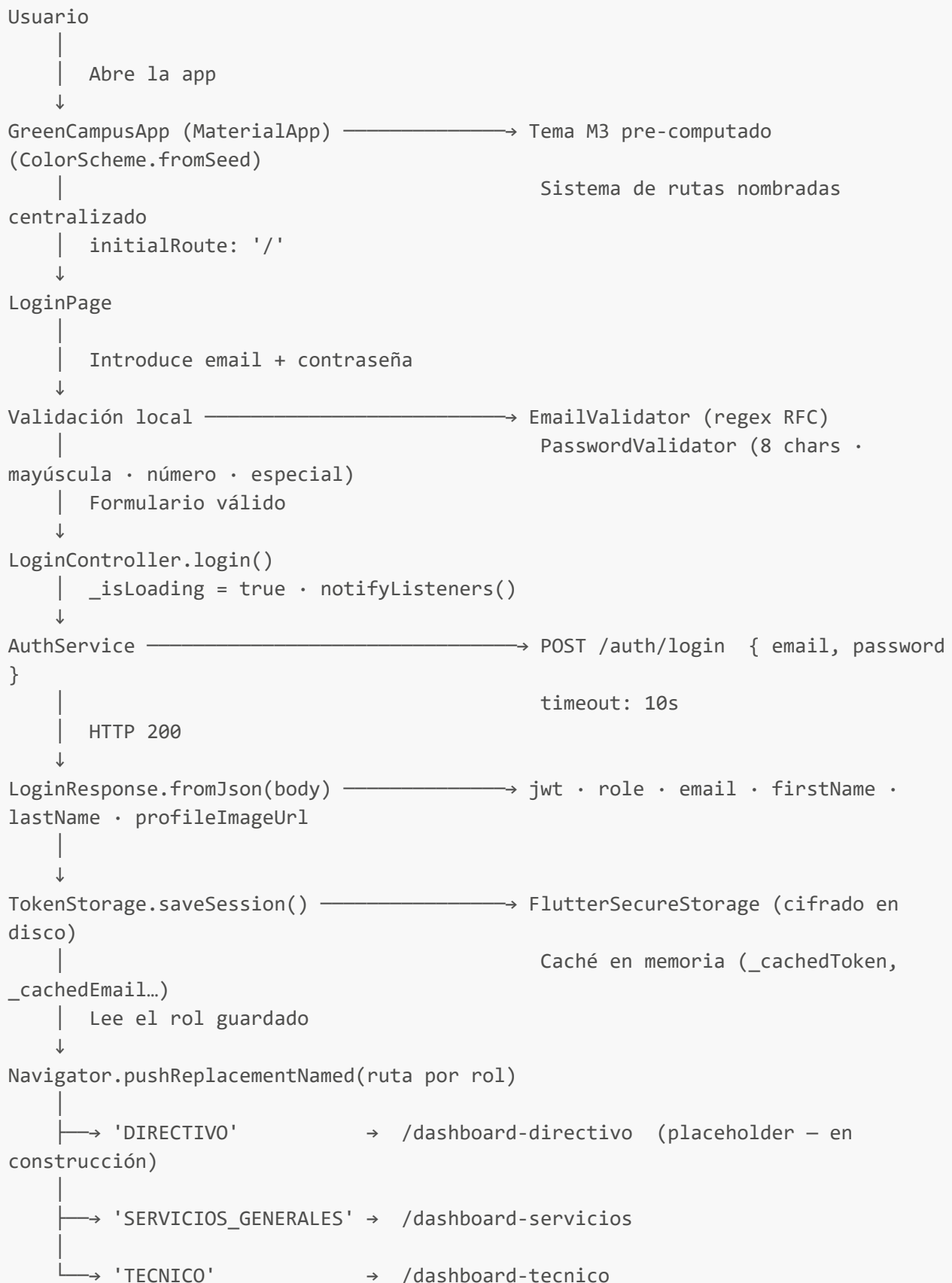
1. El sistema navega a `NotificationsPage`.
2. La pantalla muestra el estado vacío: icono de campana, texto "Sin notificaciones" y descripción "Las alertas del sistema aparecerán aquí cuando se generen."

Nota: Este caso de uso está parcialmente implementado. La estructura visual está completa pero no existe `NotificationsController` ni `NotificationsService`. La integración con el backend y la lógica de lectura/marcado de notificaciones están pendientes.

Postcondición: El usuario visualiza la pantalla de notificaciones (actualmente siempre vacía).

20. ESQUEMAS VISUALES DEL FLUJO DE LA APLICACIÓN

20.1 Flujo de arranque y autenticación



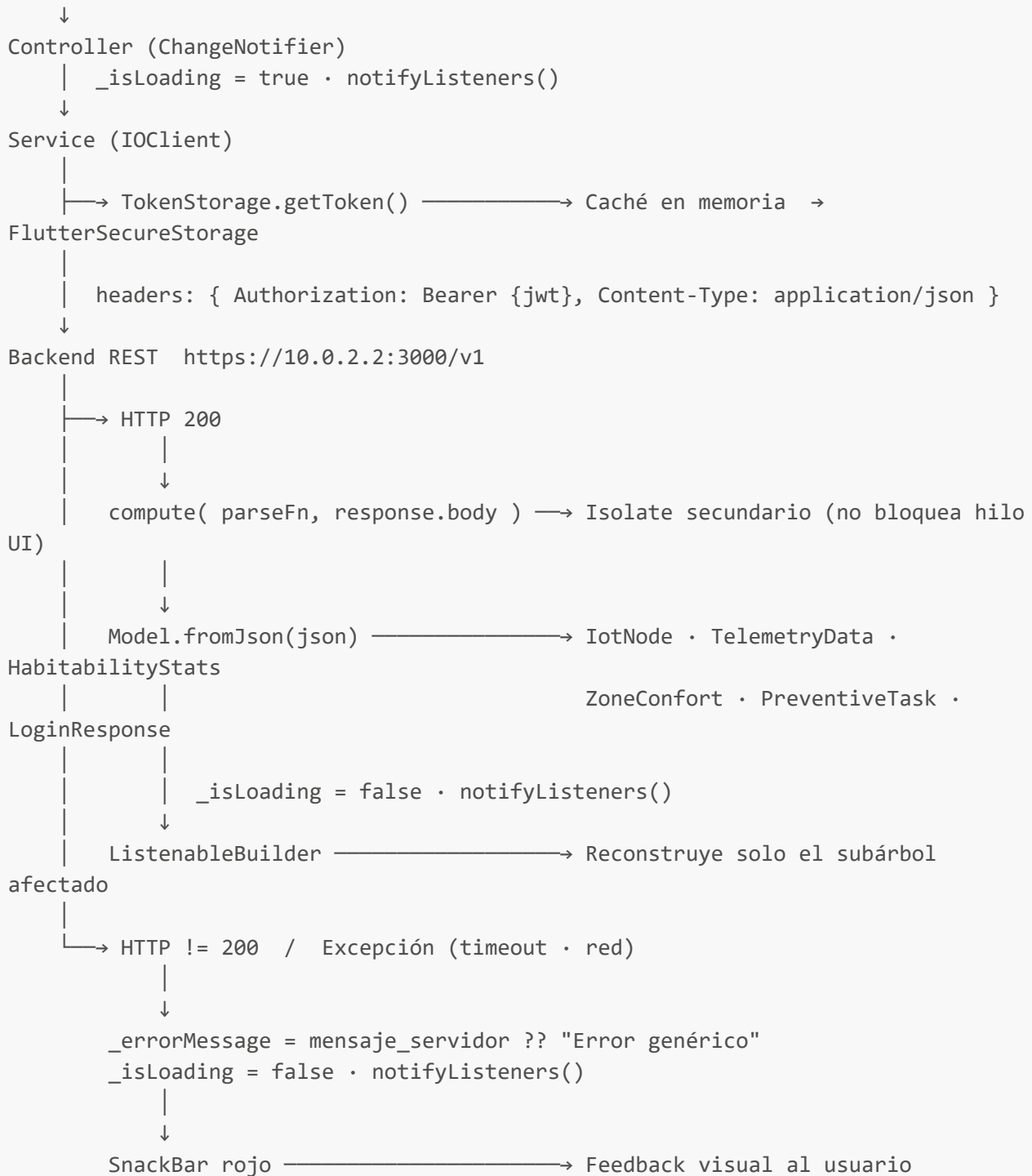
20.2 Flujo de recuperación de contraseña



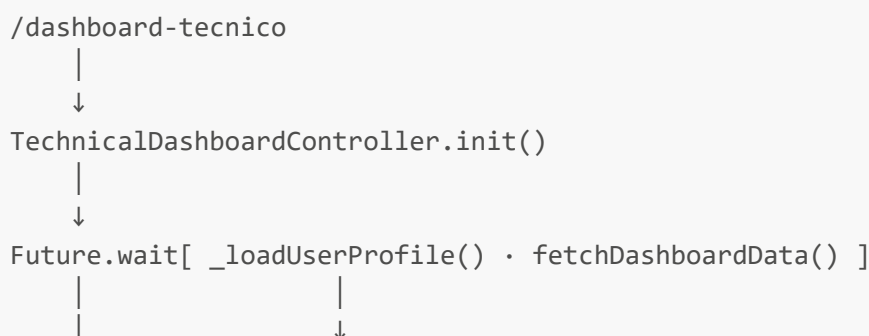
20.3 Ciclo de vida de una petición HTTP autenticada

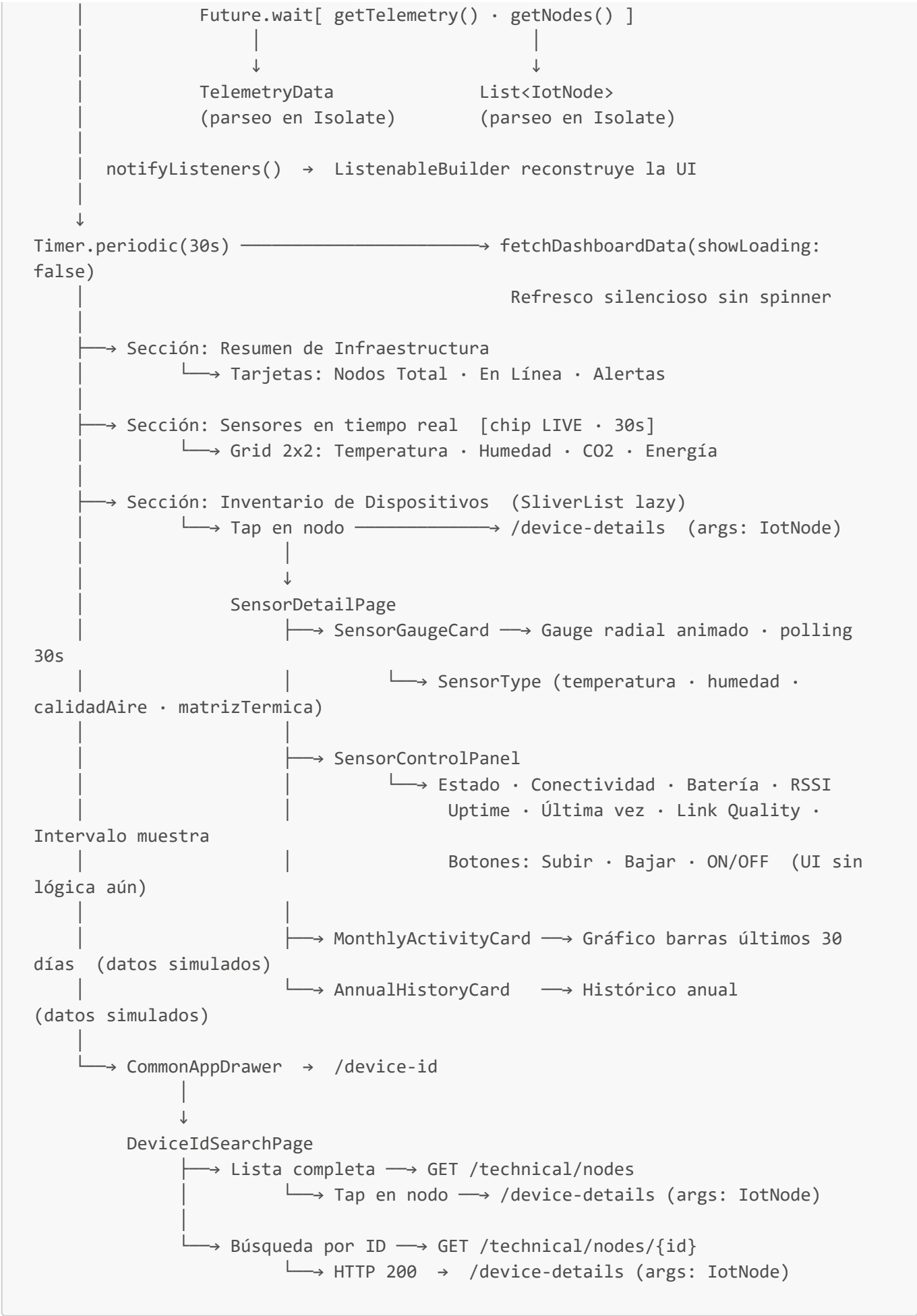
Page / Widget

Evento: tap · submit · pull-to-refresh · Timer.periodic(30s)

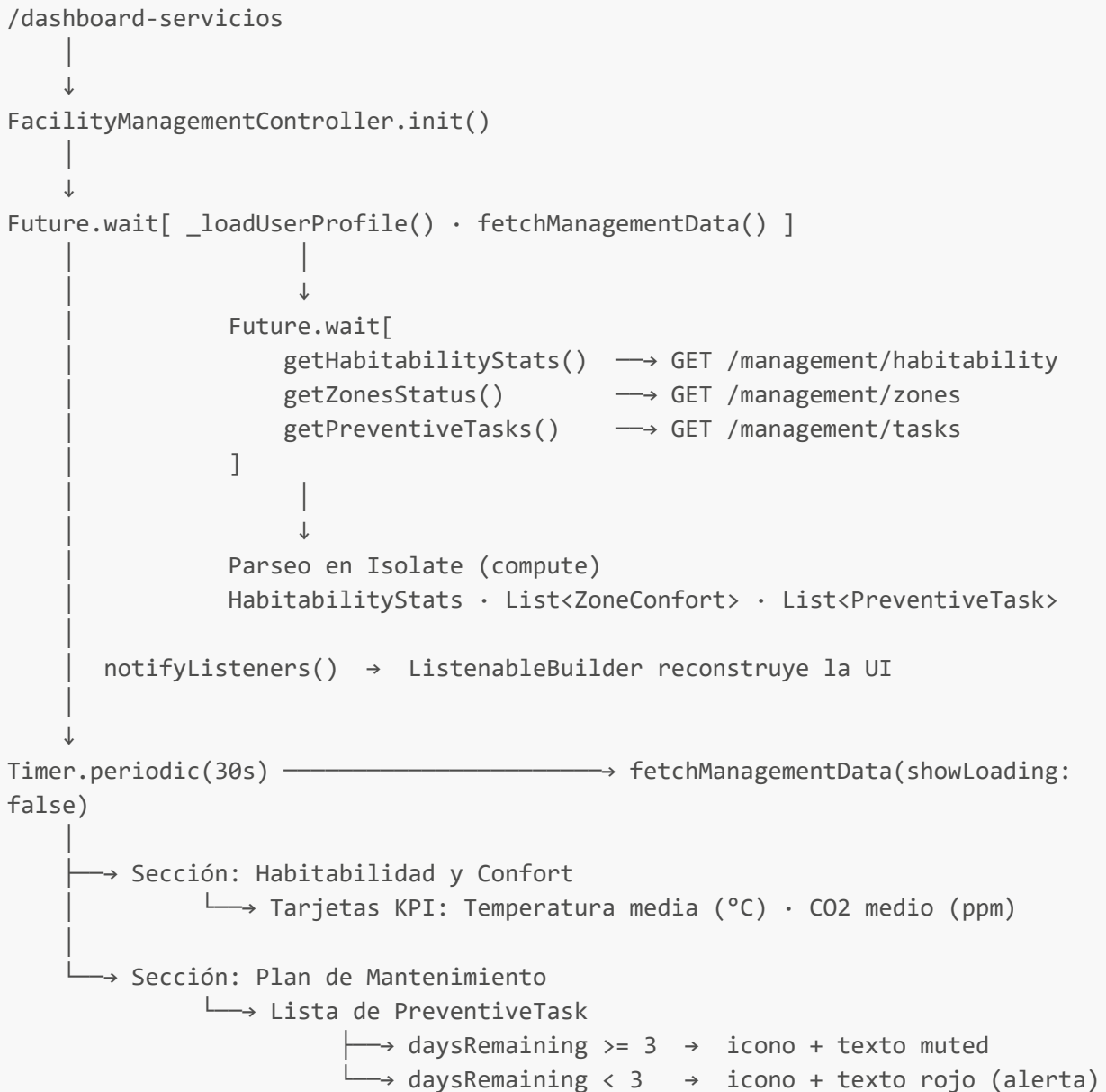


20.4 Dashboard del Técnico — flujo y sub-navegación





20.5 Dashboard de Servicios Generales — flujo

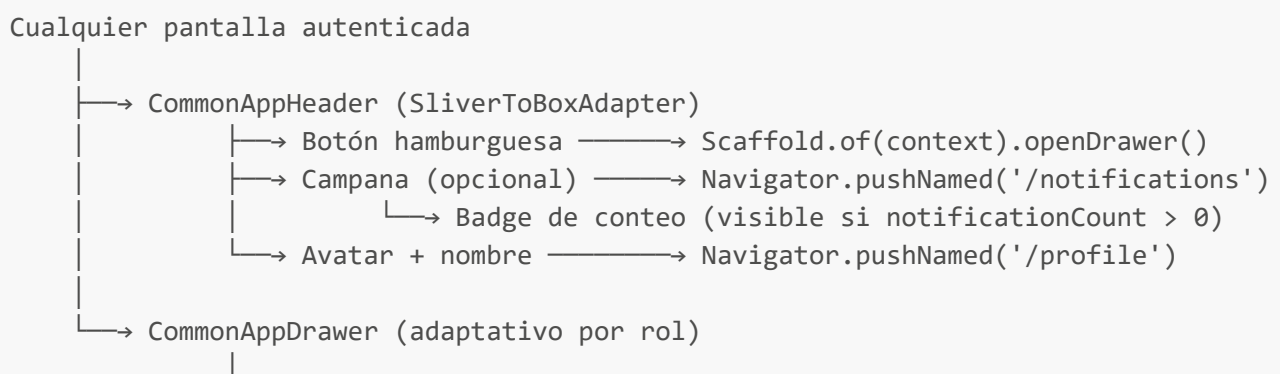


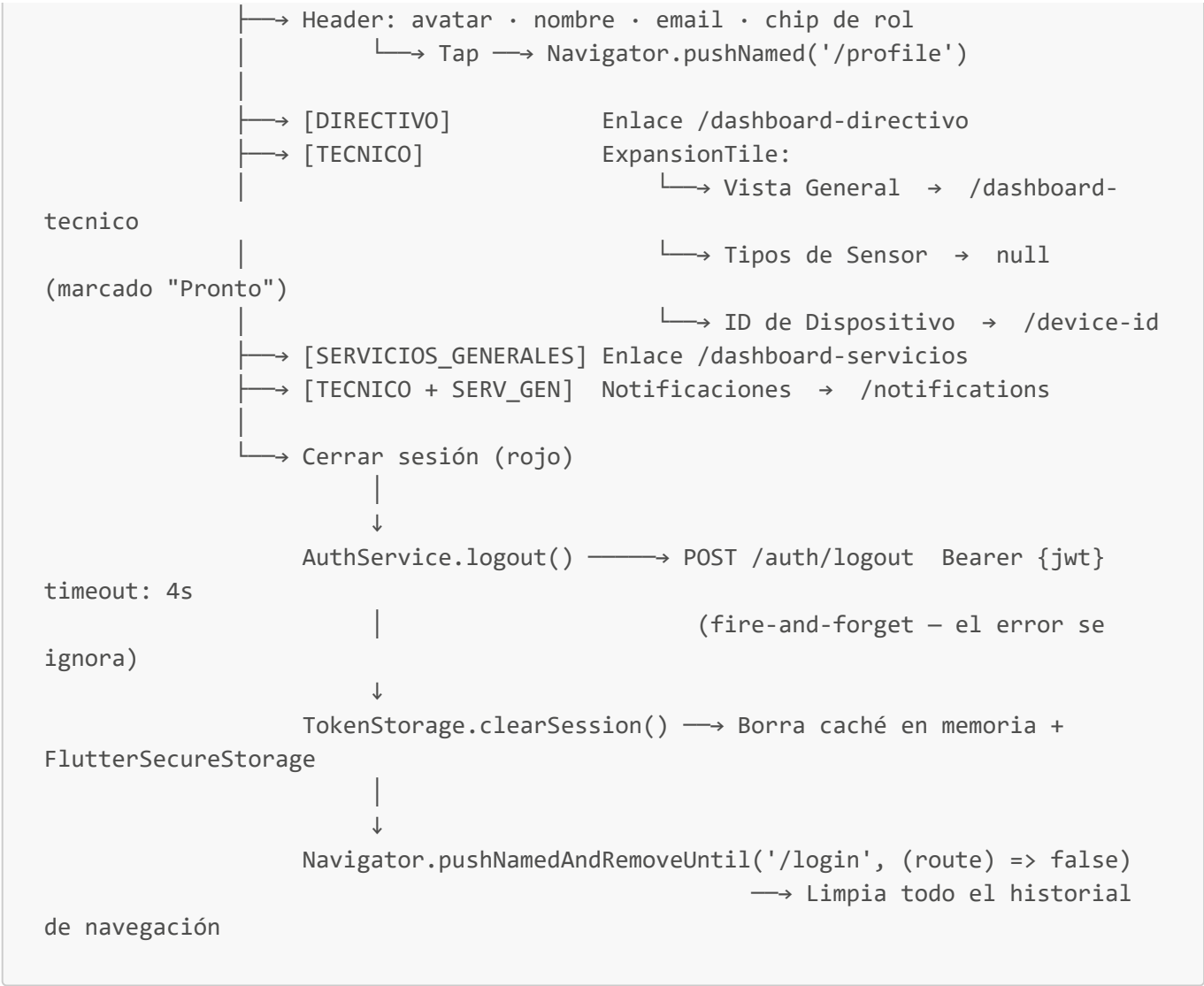
20.6 Módulo de perfil — flujo de lectura y edición





20.7 Componentes compartidos y cierre de sesión





Documento generado a partir del análisis estático completo del código fuente de la aplicación Flutter GreenCampus Mobile (directorio [Lib/](#)). El proyecto está en desarrollo activo y algunos módulos están incompletos o contienen datos simulados a la espera de la integración con el backend.